

Control I—Expressions and Statements

9.1	Expressions	403
9.2	Conditional Statements and Guards	410
9.3	Loops and Variations on WHILE	417
9.4	The GOTO Controversy and Loop Exits	420
9.5	Exception Handling	423
9.6	Case Study: Computing the Values of Static Expressions in TinyAda	432

In Chapter 8, we discussed basic and structured abstraction of data through the use of data types. In this chapter, we discuss the basic and structured abstraction of control through the use of expressions and statements. Structured control through the use of procedures and functions, and the organization of run-time environments, will be studied in the next chapter. As noted in Chapter 1, unit-level data and control converge to the same kinds of language mechanisms, and they will be studied in the chapter on modules.

Expressions represent the most fundamental computation mechanism in programs. An **expression**, in its pure, mathematical, form, returns a value and produces no side effect. That is, it does not change program memory. By contrast, a **statement** is executed for its side effects and returns no value. Many languages, however, do not make such a clear distinction and allow expressions to contain side effects. In functional languages—sometimes also called **expression languages**—virtually all language constructs are expressions. Even in nonfunctional languages, expressions play a significant role, and in some languages, such as C (which could be called an **expression-oriented language**), expressions make up a much larger portion of the language than statements. Expressions are also the parts of programming languages that are closest in appearance to mathematics. In the absence of side effects, program expressions have semantics that are very similar to those of mathematical expressions, which leads to semantic simplicity and precision. Unfortunately, in the presence of side effects, expressions can behave in ways that are very different from their mathematical counterparts. This can be a source of confusion and error. In fact, the semantics of expressions with side effects have a significant control component. The way that such expressions are evaluated, including the order in which subexpressions are computed, can have a major impact on their meaning, something that is never true for mathematical expressions. That is why we discuss them in this chapter, along with more explicit forms of control.

Explicit control structures first appeared in programming languages as GOTOs, which are simple imitations of the jump statements of assembly language, transferring control directly, or after a test, to a new location in the program. With Algol60 came the improvement of **structured control**, in which control statements transfer control to and from sequences of statements that are (at least in principle) **single-entry, single-exit**, that is, those that enter from the beginning and exit from the end. Examples of such single-entry, single-exit constructs are the **blocks** of Algol60, Algol68, C, and Ada, which may also include declarations, and which you studied in Chapter 7.¹

Structured programming led to an enormous improvement in the readability and reliability of programs. It's no surprise, then, that structured control constructs are part of most major languages today. Some languages do away with GOTOs altogether, although a (now somewhat muted) debate continues to this day on the utility of GOTOs within the context of structured programming.

¹All these languages contain certain relaxations of the single-entry, single-exit property of these constructs, but the principle of controlling entry and exit points remains.

In this chapter, we will first discuss expressions and the variety of control questions that can arise during their evaluation. We then discuss structured control mechanisms (the assignment statement was already studied in Chapter 7), after which we review the GOTO issue. In the last section of this chapter, we begin a discussion of exception handling, which is essentially a preemptive control mechanism, causing the explicit control structure of a program to be disturbed. In that section, we discuss the appearance and operation of exceptions in several languages; implementation issues are postponed to the next chapter, since they involve the runtime environment.

9.1 Expressions

As you saw in Chapters 6 and 7, basic expressions are literals (manifest constants) and identifiers. More complex expressions are built up recursively from basic expressions by the application of operators and functions, sometimes involving grouping symbols such as parentheses. For example, in the simple arithmetic expression $3 + 4 * 5$, the **+** operator is applied to its two **operands**, consisting of the integer literal 3 and the (sub)expression $4 * 5$, which in its turn represents the application of the ***** operator to its operands, consisting of the integer literals 4 and 5. Operators can take one or more operands: An operator that takes one operand is called a **unary operator**; an operator that takes two is a **binary operator**. Operators can be written in **infix**, **postfix**, or **prefix** notation, corresponding to an inorder, postorder, or preorder traversal of the syntax tree of the expression. For example, as you learned in Chapter 6, the infix expression $3 + 4 * 5$ with the syntax tree shown in Figure 9.1 is written in postfix form as $3\ 4\ 5\ *\ +$ and in prefix form as $+ \ 3\ *\ 4\ 5$.

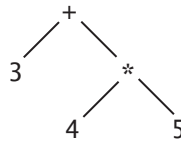


Figure 9.1 Syntax tree for infix expression $3 + 4 * 5$

The advantage of postfix and prefix forms is that they do not require parentheses to express the order in which operators are applied. Thus, operator precedence is not required to disambiguate an unparenthesized expression. For instance, $(3 + 4) * 5$ is written in postfix form as $3\ 4\ +\ 5\ *$ and in prefix form as $*\ +\ 3\ 4\ 5$. Associativity of operators is also expressed directly in postfix and prefix form without the need for a rule. For example, the postfix expression $3\ 4\ 5\ +\ +$ right associates and $3\ 4\ +\ 5\ +$ left associates the infix expression $3 + 4 + 5$.

Many languages make a distinction between operators and functions. Operators are predefined and (if binary) written in infix form, with specified associativity and precedence rules, whereas functions, which can be either predefined or user-defined, are written in prefix form and the operands are viewed as **arguments** or **actual parameters** to calls of the functions. For example, in C, if we wrote the expression $3 + 4 * 5$ with user-defined addition and multiplication operations, it would appear as `add(3, mul(4, 5))`. In fact, the distinction between operators and functions is arbitrary, since they are equivalent concepts. Historically, however, the distinction *was* significant, since built-in operators were implemented using highly optimized **inline code** (code for the function body that is inserted directly at

the point where the function would be called),² while function calls required the building of sometimes time-consuming **activations** (described in the next chapter). Modern translators, however, often inline even user-defined functions; in any case, the performance penalty of activations has largely disappeared in modern architectures.

Programming languages, too, have increasingly allowed programmers to overload the built-in operators and even to define new infix operators with arbitrary associativity and precedence, as you saw in Sections 3.4 and 5.4. Some languages even allow all operators to be written in either prefix or infix form. For example, $3 + 4 * 5$ can be written just as well in Ada as `"+"(3, "*" (4, 5))`. A few languages use either the prefix or postfix form exclusively, for both predefined and user-defined operations. For example, stack-based languages such as PostScript and FORTH use postfix notation exclusively, while Lisp uses prefix notation. Lisp also requires expressions to be **fully parenthesized**; that is, all operators and operands must be enclosed in parentheses. This is because LISP operators can take variable numbers of arguments as operands. Thus, $3 + 4 * 5$ and $(3 + 4) * 5$ would look as follows in LISP:

```
(+ 3 (* 4 5))
(* (+ 3 4) 5)
```

Each programming language has rules for evaluating expressions. According to one common evaluation rule, all operands are evaluated first and then operators are applied to them. This rule, called **applicative order evaluation**, or sometimes **strict evaluation**, is the most common rule in programming languages. It corresponds to a bottom-up evaluation of the values at nodes of the syntax tree representing an expression. For example, the expression $(3 + 4) * (5 - 6)$ is represented by the syntax tree shown in Figure 9.2.

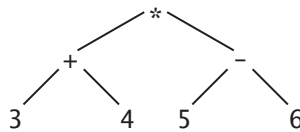


Figure 9.2 Syntax tree for the expression $(3 + 4) * (5 - 6)$

In applicative order evaluation, first the “+” and “−” nodes are evaluated to 7 and −1, respectively, and then the “*” is applied to get −7.

Let us also consider how this would appear in terms of user-defined functions, written in prefix form in C syntax:

```
mul(add(3, 4), sub(5, 6))
```

Applicative order says evaluate the arguments first. Thus, calls to `add(3, 4)` and `sub(5, 6)` are made, which are also evaluated using applicative order. Then, the calls are replaced by their returned values, which are 7 and −1. Finally, the call

```
mul(7, -1)
```

²Code inlining is an important optimization technique for translators; indeed, C++ has an `inline` keyword to allow the programmer to specifically request inlining, if possible; see Chapter 6. Inlining of operators and nonrecursive functions with no nonlocal references is relatively easy, but in the most general case inlining may be difficult or impossible. For an example of the difficulties of inlining, see Exercise 11.14.

is made. Note that this process corresponds to a bottom-up “reduction” of the syntax tree to a value. First the $+$ and $-$ nodes are replaced by their values 7 and -1 , and finally the root $*$ node is replaced by the value -7 , which is the result of the expression.

This raises the question of the order in which the subexpressions $(3 + 4)$ and $(5 - 6)$ are computed, or the order in which the calls `plus(3, 4)` and `minus(5, 6)` are made. A natural order is left to right, corresponding to a left-to-right traversal of the syntax tree. However, many languages explicitly state that there is no specified order for the evaluation of arguments to user-defined functions. The same is often true for predefined operators as well.

There are several reasons for this. One is that different machines may have different requirements for the structure of calls to procedures and functions. Another is that a translator may attempt to rearrange the order of computation so that it is more efficient. For example, consider the expression $(3 + 4) * (3 + 4)$. A translator may discover that the same subexpression, namely, $3 + 4$, is used twice and will evaluate it only once. And in evaluating a call such as `max(3, 4+5)`, a translator may evaluate $4 + 5$ before 3 because it is more efficient to evaluate more complicated expressions first.

If the evaluation of an expression causes no side effects, then the expression will yield the same result, regardless of the order of evaluation of its subexpressions. In the presence of side effects, however, the order of evaluation can make a difference. Consider, for example, the C program of Figure 9.3.

```
(1) #include <stdio.h>

(2) int x = 1;

(3) int f(){
(4)     x += 1;
(5)     return x;
(6) }

(7) int p(int a, int b) {
(8)     return a + b;
(9) }
(10) main(){
(11)     printf("%d\n", p(x, f()));
(12)     return 0;
(13) }
```

Figure 9.3 C Program showing evaluation order matters in the presence of side effects

If the arguments of the call to `p` on line 11 are evaluated left to right, this program will print 3. If the arguments are evaluated right to left, the program will print 4. The reason is that a call to the function `f` has a side effect: It changes the value of the global variable `x`.

In a language that explicitly states that the order of evaluation of expressions is unspecified, programs that depend on the order of evaluation for their results (such as the one above) are incorrect, even though they may have predictable behavior for one or more translators.

In spite of the problems side effects cause in expression evaluation, sometimes expressions are explicitly constructed with side effects in mind. For example, in C, assignment (a quintessential side effect) is an expression, not a statement: `x = y` not only assigns the value of `y` to `x` but also returns the copied value as its result.³ Thus, assignments can be combined in expressions, so that

```
x = (y = z)
```

assigns the value of `z` to both `x` and `y`. Note that in languages with this kind of construction, assignment can be considered to be a binary operator similar to arithmetic operators. In that case its precedence is usually lower than other binary operators, and it is made right associative. Thus,

```
x = y = z + w
```

in C assigns the sum of `z` and `w` to both `x` and `y` (and returns that value as its result).

C and other expression languages also have a **sequence operator**, which allows several expressions to be combined into a single expression and evaluated sequentially. In C, the sequence operator is the comma operator, which has precedence lower than any other operator. Unlike most other operators in C, the order of evaluation is specified as left to right, and the value of the rightmost expression is the value returned by the entire expression. For example, given that `x` and `y` are integer variables with values 1 and 2, respectively, the C expression

```
x = (y+=1, x+=y, x+1)
```

returns the value 5 and leaves `x` with the value 5 and `y` with the value 3.

The evaluation of an expression can sometimes be performed even without the evaluation of all its subexpressions. An interesting case is that of the Boolean, or logical, expressions. For example, the Boolean expressions (in Ada or C++ syntax)

```
true or x
```

and

```
x or true
```

are true regardless of whether `x` is true or false. Similarly,

```
false and x
```

and

```
x and false
```

are clearly false regardless of the value of `x`. In a programming language, one can specify that Boolean expressions are to be evaluated in left-to-right order, up to the point where the truth value of the entire expression becomes known and then the evaluation stops. A programming language that has this rule is said to possess **short-circuit evaluation** of Boolean or logical expressions.

³When a type conversion occurs, the type of the copied value is that of `x`, not `y`.

Short-circuit evaluation has a number of benefits. One is that a test for the validity of an array index can be written in the same expression as a test of its subscripted value, as long as the range test occurs first:

```
if (i <= lastindex and a[i] >= x) ... // C++ code
```

Without short-circuit evaluation, the test `i <= lastindex` will not prevent an error from occurring if `i > lastindex`, since the expression `a[i]` in the second part of the expression will still be computed. (This assumes, of course, that an out-of-range array subscript *will* produce an error.) Without short-circuit evaluation, the test must be written using nested `if`'s:

```
if (i <= lastindex) if (a[i] >= x) ...
```

Similarly, a test for a null pointer can be made in the same expression as a dereference of that pointer, using short-circuit evaluation:

```
if (p != 0 and p->data == x) ... // C++ code
```

Note that the order of the tests becomes important in short-circuit evaluation. Short-circuit evaluation will protect us from a runtime error if we write:

```
// ok: y % x requires x != 0
if (x != 0 and y % x == 0) ...
```

but not if we write:

```
if (y % x == 0 and x != 0) ... // not ok!
```

Most languages require Boolean operators to be short-circuit. C/C++, Java, and ML all have short-circuit Boolean operators (but not Pascal!). In Ada, which offers both kinds, the keywords `and` and `or` are not short-circuit, but the operators (two keywords each) “`and then`” and “`or else`” *are* short-circuit. For example:

```
-- Ada short-circuit code:
if (i <= lastindex) and then (a(i) = x) then ...
```

While we are on the subject of Boolean operators, we note the unfortunate lack of agreement on the names of these operators. C and Java use the somewhat obscure notation `&&` for `and`, `||` for `or`, and `!` for `not`. C++ allows these same operators but also allows the more common keywords `and`, `or`, and `not`. ML's names for the binary logical operators are `andalso` and `orelse`, presumably to call attention to their short-circuit nature. (ML has another, completely separate use for the keyword `and`, and the keyword `or` does not exist in ML.) Other languages occasionally use imitations of the operations as they are written in mathematical logic. For example, `/\` (forward slash followed by backslash) is used for `and`, `\/` (the reverse of `/\`) is used for `or`, and `~` (tilde) is used for `not`.

Boolean expressions are not the only expressions whose subexpressions may not all be evaluated: Many languages have expressions that mimic control statements but return values; two of these are **if expressions** and **case expressions**.

An `if` (or `if-then-else`) operator is a **ternary operator**—that is, it has *three* operands. These operands are as follows: the conditional test expression (which is typically Boolean), the “then” expression (whose value is the result of the entire expression if the test expression is true), and the “else” expression (whose value is the result of the entire expression if the test expression is false). As a prefix function it would be written as:

```
if ( test-exp, then-exp, else-exp )
```

but most languages use a **mix-fix** form that distributes parts of the syntax of the operator throughout the expression in traditional style. For example, in ML the `if`-expression appears as:

```
if e1 then e2 else e3
```

while in C the same expression appears as:

```
e1 ? e2 : e3
```

Note that an `if`-expression may not have an optional `else`-part (unlike an `if`-statement), since there would be no value to return if the test expression were to evaluate to false.

`If`-expressions *never* have all of their subexpressions evaluated. That is, `e1` (the test expression) is *always* evaluated first, but, based on that value, only one of `e2` and `e3` is ever evaluated. This is the only reason for the `if`-expression to exist at all.

ML and some other functional languages also have a **case expression**:

```
case e1 of
  a => e2 |
  b => e3 |
  c => e4 ;
```

which is more or less equivalent to a series of nested `if`-expressions: `if e1 = a then e2 else if e1 = b then e3 . . .`. Case-expressions have some special properties, however, and were discussed in Chapter 3.

Interestingly, the short-circuit Boolean operations can be defined using `if`-expressions as follows:

```
e1 and e2 = if e1 then e2 else false (* short circuit and *)
e1 or e2 = if e1 then true else e2   (* short circuit or *)
```

Short-circuit Boolean and `if` operators are a special case of operators that **delay** evaluating their operands. The general situation is called **delayed evaluation**, or sometimes **nonstrict evaluation**. In the case of the short-circuit operators, `e1` and `e2` and `e1 or e2`, both delay the evaluation of `e2` until `e1` is evaluated. Similarly, the `if` operator delays the evaluation of both `e2` and `e3` until `e1` is evaluated.

It is worth restating the fact that, in the absence of side effects (changes to variables in memory, input and output), the order of evaluation of expressions is immaterial to the final value of the expression.⁴ In fact, in a language in which side effects do not exist or are carefully controlled, such as the pure

⁴As long as it actually *has* a value; see Chapter 8.

functional languages (Chapter 3), expressions in programs share an important property with mathematical expressions. We call this shared property the **substitution rule** (or sometimes **referential transparency**). According to this rule, any two expressions in a program that have the same value may be substituted for each other anywhere in the program. In other words, their values always remain equal, regardless of the context in which they are evaluated. For example, if x and y have the same value at one point in a referentially transparent program, then they *always* have the same value⁵ and may be freely substituted for each other anywhere. (Note that this prohibits x and y from being variables in the programming language sense. Instead, they must be constants.)

Referential transparency allows for a very strong form of delayed evaluation to be used that has important theoretical and practical consequences. It is called **normal order evaluation**. A precise mathematical definition can be given in the context of the lambda calculus, as was explained briefly in Chapter 3. For our purposes in this section, normal order evaluation of an expression means that each operation (or function) begins its evaluation *before* its operands are evaluated, and each operand is evaluated only if it is needed for the calculation of the value of the operation.

As an example of normal order evaluation, consider the following functions (in C syntax):

```
int double(int x){
    return x + x;
}

int square(int x){
    return x * x;
}
```

Now consider the expression `square(double(2))`. Using normal order evaluation, the call to `square` is replaced by `double(2)*double(2)` without evaluating `double(2)`, and then `double(2)` is replaced by `2 + 2`, so that the following sequence of replacements occurs:

$$\text{square}(\text{double}(2)) \Rightarrow \text{double}(2) * \text{double}(2) \Rightarrow (2 + 2) * (2 + 2)$$

Only after these replacements occur is the expression evaluated, typically in left-to-right order:

$$(2 + 2) * (2 + 2) \Rightarrow 4 * (2 + 2) \Rightarrow 4 * 4 \Rightarrow 16$$

Note how this differs from applicative order evaluation, which evaluates the call to `double` before the call to `square`, and would result in the following sequence of replacements (with evaluation intermingled):

$$\text{square}(\text{double}(2)) \Rightarrow \text{square}(2 + 2) \Rightarrow \text{square}(4) \Rightarrow 4 * 4 \Rightarrow 16$$

Normal order evaluation implements a kind of code inlining, in which the bodies of functions are substituted at their call sites *before* evaluation occurs; see footnote 2.

⁵As long as we are in the scope of a particular declaration of these variables. Obviously, different declarations may cause different values to be associated with these names.

In the absence of side effects, normal order evaluation does not change the semantics of a program. While it might seem inefficient (e.g., $2+2$ gets evaluated twice instead of once in the previous example), it can be made efficient, and it has a number of good properties that make it useful, particularly for functional languages. As a simple example, if C used normal order evaluation, the C if-expression $e1 \text{ ? } e2 \text{ : } e3$ can be written (for each data type) as an ordinary C function, instead of needing special syntax in the language:

```
int if_exp(int x, int y, int z){
    if (x) return y; else return z;
}
```

Here y and z are only evaluated if they are needed in the code of `if_exp`, which is the behavior that we want.

On the other hand, the presence of side effects can mean that normal order evaluation substantially changes the semantics of a program. Consider, for example, the C function:

```
int get_int(){
    int x;
    /* read an integer value into x from standard input */
    scanf("%d",&x);
    return x;
}
```

This function has a side effect (input).

Now consider what would happen to the expression `square(get_int())` using normal order evaluation: It would be expanded into `get_int()*get_int()`, and *two* integer values, rather than just one, would be read from standard input—a very different outcome from what we would expect.

Normal order evaluation appears as so-called **lazy evaluation** in the functional language Haskell, as was studied in detail in Chapter 3. Normal order evaluation appears also as a parameter passing technique for functions in Algol60, where it is known as **pass by name**; this will be studied briefly in the next chapter.

9.2 Conditional Statements and Guards

The most typical form of structured control is execution of a group of statements only under certain conditions. This involves making a Boolean, or logical, test before entering a sequence of statements. The well-known **if-statement** is the most common form of this construct. The various ways such a conditional can be introduced will be discussed shortly.

First, however, we want to describe a general form of conditional statement that encompasses all the various conditional constructs. This general form, known as the **guarded if** statement, was introduced by E. W. Dijkstra, and looks like this:

```
if B1 -> S1
|  B2 -> S2
|  B3 -> S3
...
|  Bn -> Sn
fi
```

The semantics of this statement are as follows: each Bi 's is a Boolean expression, called a **guard**, and each Si 's is a statement sequence. If one Bi 's evaluates to true, then the corresponding statement sequence Si is executed. If more than one Bi 's is true, then the one and only corresponding Si 's is selected for execution. If no Bi is true, then an error occurs.

There are several interesting features in this description. First, it does not say that the first Bi that evaluates to true is the one chosen. Thus, the guarded if introduces nondeterminism into programming, a feature that becomes very useful in concurrent programming (Chapter 13). Second, it leaves unspecified whether all the guards are evaluated. Thus, if the evaluation of a Bi has a side effect, the result of the execution of the guarded if may be unknown. Of course, the usual deterministic implementation of such a statement would sequentially evaluate the Bi 's until a true one is found, whence the corresponding Si is executed and control is transferred to the point following the guarded statement.

The two major ways that programming languages implement conditional statements like the guarded if are as if-statements and case-statements.

9.2.1 If-Statements

The basic form of the if-statement is as given in the extended Backus-Naur form (EBNF) rule for the if statement in C, with an optional “else” part:

$$\text{if-statement} \rightarrow \text{if (expression) statement [else statement]}$$

where *statement* can be either a single statement or a sequence of statements surrounded by braces:

```
if (x > 0) y = 1/x;
else{
    x = 2;
    y = 1/z;
}
```

This form of the if (which also exists in Java and Pascal) is problematic, however, because it is ambiguous in the syntactic sense described in Chapter 6. Indeed, the statement

```
if (e1) if (e2) S1 else S2
```

has two different parse trees according to the BNF. These parse trees are shown in Figure 9.4.

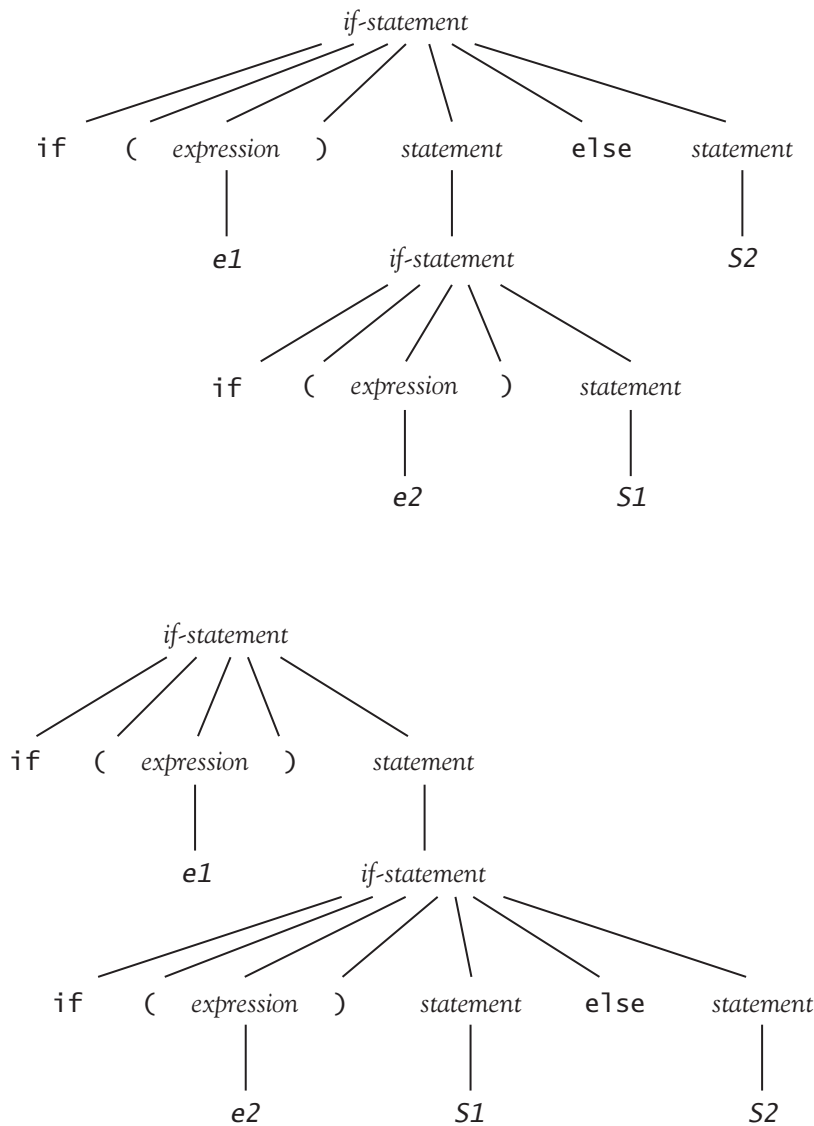


Figure 9.4 Two parse trees for the statement `if (e1) if (e2) S1 else S2`

This ambiguity is called the **dangling-else** problem. The syntax does not tell us whether an `else` after two `if`-statements should be associated with the first or second `if`. C and Pascal solve this problem by enforcing a **disambiguating rule**, which works as follows: The `else` is associated with the closest preceding `if` that does not already have an `else` part. This rule is also referred to as the **most closely nested rule** for `if`-statements. It states that the second parse tree is the correct one.

The fact that the dangling-else problem arises at all indicates an essential language design problem. This is true for two reasons. First, it makes us state a new rule to describe what is essentially a syntactic feature. Second, it makes the interpretation of the `if`-statement by the reader more difficult; that is, it

violates the readability design criterion. As an illustration, if we want actually to associate the `else` with the first `if` in the preceding statement, we would have to write either:

```
if ( e1 ) { if ( e2 ) S1 } else S2
```

or:

```
if ( e1 ) if ( e2 ) S1 else ; else S2
```

There are other ways to solve the dangling else than by using a disambiguating rule. In fact, it is possible to write out BNF rules that specify the association precisely, but these rules make the grammar somewhat more complex. (Java does this—see Exercises 9.28 and 9.29.) A better way is to use a **bracketing keyword** for the if-statement, such as the Ada rule:

$$\text{if-statement} \rightarrow \text{if condition then sequence-of-statements} \\ \text{[else sequence-of-statements] end if ;}$$

The two bracketing keywords `end if` (together with the semicolon) close the if-statement and remove the ambiguity, since we must now write either:

```
if e1 then if e2 then S1 end if ; else S2 end if;
```

or:

```
if e1 then if e2 then S1 else S2 end if ; end if ;
```

to decide which `if` is associated with the `else` part. This also removes the necessity of using brackets (or a `begin - end` pair) to open a new sequence of statements. The if-statement can open its own sequence of statements and, thus, becomes fully structured:

```
if x > 0.0 then
  y := 1.0/x;
  done := true;
else
  x := 1.0;
  y := 1.0/z;
  done := false;
end if;
```

A similar approach was taken in the historical (but influential) language Algol68, which suggestively uses keywords spelled backward as bracketing keywords:

```
if e1 then S1 else S2 fi
```

Thus, the bracketing keyword for if-statements is `fi`. (One still sees this convention in theoretical language design publications—witness the guarded if of Dijkstra, described above.)

An extension of the if-statement simplifies things when there are many alternatives. Multiple `else`'s statements would have to be written in Ada as:

```

if e1 then S1
else if e2 then S2
else if e3 then S3
end if ; end if ; end if;

```

with many “end if”s piled up at the end to close all the if-statements. Instead, the “else if” is compacted into a new reserved word `elsif` that opens a new sequence of statements at the same level:

```

if e1 then S1
elsif e2 then S2
elsif e3 then S3
end if ;

```

An additional issue regarding the if-statement is the required type of controlling expression (written in the preceding examples variously as *Bi* or *ei*, indicating possibly Boolean or not). In Java and Ada (as well as Pascal), the test must always have the Boolean type. C has no Boolean type, and in C (as well as C++), the controlling expression can have either the integral type (see Figure 9.3) or the pointer type. The resulting value is then compared to 0 (the null pointer in the case of pointer type); unequal comparison is equivalent to true, equal comparison is equivalent to false. Thus, in C a test such as:

```
if (p != 0 && p->data == x) ...
```

can be written as:

```
if (p && p->data == x) ...
```

9.2.2 Case- and Switch-Statements

The case-statement (or switch-statement in C/C++/Java) was invented by C. A. R. Hoare as a special kind of guarded if, where the guards, instead of being Boolean expressions, are ordinal values that are selected by an ordinal expression. An example in C is given in Figure 9.5.

```

(1) switch (x - 1){
(2)     case 0:
(3)         y = 0;
(4)         z = 2;
(5)         break;
(6)     case 2:
(7)     case 3:
(8)     case 4:
(9)     case 5:
(10)        y = 3;
(11)        z = 1;
(12)        break;

```

Figure 9.5 An example of the use of the switch statement in C (*continues*)

(continued)

```

(13)  case 7:
(14)  case 9:
(15)      z = 10;
(16)      break;
(17)  default:
(18)      /* do nothing */
(19)      break;
(20) }
```

Figure 9.5 An example of the use of the switch statement in C

The semantics of this statement are to evaluate the controlling expression ($x - 1$ on line 1 in Figure 9.5), and to transfer control to the point in the statement where the value is listed. These values must have integral types in C (and conversions occur, if necessary, to bring the types of the listed cases into the type of the controlling expression). No two listed cases may have the same value after conversion. Case values may be literals as in the above example, or they can be constant expressions as C defines them (that is, compile-time constant expressions with no identifiers; see Chapter 7). If the value of the controlling expression is not listed in a case label, then control is transferred to the `default` case (line 17 in Figure 9.5), if it exists. If not, control is transferred to the statement following the `switch` (that is, the switch statement **falls through**).

The structure of this statement in C has a number of somewhat novel features. First, the case labels are treated syntactically as ordinary labels, which allows for potentially bizarre placements; see Exercise 9.30. Second, without a `break` statement, execution falls through to the next case. This allows cases to be listed together without repeating code (such as the cases 2 through 5, lines 6–9 in Figure 9.5); it also results in incorrect execution if a `break` statement is left out.

Java has a `switch` statement that is virtually identical to that of C, except that it tightens up the positioning of the case labels to eliminate bizarre situations.

A somewhat more “standard” version of a case statement (from the historical point of view) is that of Ada. The C example of Figure 9.5 is written in Ada in Figure 9.6.

```

(1)  case x - 1 is
(2)      when 0 =>
(3)          y := 0;
(4)          z := 2;
(5)      when 2 .. 5 =>
(6)          y := 3;
(7)          z := 1;
(8)      when 7 | 9 =>
(9)          z := 10;
(10)  when others =>
(11)      null;
(12)  end case;
```

Figure 9.6 An example of the use of the case statement in Ada, corresponding to the C example of Figure 9.5

In Ada, case values can be grouped together, either by listing them individually, separated by vertical bars (as in line 8 of Figure 9.6), or by listing a range (such as 2 . . 5 in line 5 of Figure 9.6). Also, in Ada case values must not only be distinct but they must also be exhaustive. If a legal value is not listed and there is no default case (called *others* in Ada, as on line 10 of Figure 9.6), a compile-time error occurs (i.e., there is no fall through as in C). An implication of this is that the complete set of possible values for the controlling expression must be known at compile time. Thus, enumerations and small subranges are typically used to control case statements.

The functional language ML also has a case construct, but it is an expression that returns a value, rather than a statement (as is usual in functional languages). An example of a function in ML whose body is a case expression is given in Figure 9.7.

```
(1) fun casedemo x =
(2)   case x - 1 of
(3)     0 => 2 |
(4)     2 => 1 |
(5)     _ => 10
(6)   ;
```

Figure 9.7 An example of a case expression in ML

The syntax of a case expression in ML is similar to that of Ada, except that cases are separated by vertical bars (and there is no *when* keyword). Further, cases in an ML case expression are patterns to be matched, rather than ranges or lists of values. In Figure 9.7, the only patterns are the numbers 0, 2, and the **wildcard pattern** (the underscore in line 5 of Figure 9.7), which functions as a default case. For example, the function definition of Figure 9.7 would result in the following evaluations in ML:

```
- casedemo 1;
val it = 2 : int
- casedemo 9;
val it = 10 : int
```

ML permits the cases of a case expression to be not exhaustive, but generates a warning in that case (since no value can be returned if an unlisted case occurs, and a runtime error must occur):

```
val casedemo = fn : int -> int
- fun casedemo x =
=   case x - 1 of
=     0 => 2 |
=     2 => 1
=   ;
stdIn:18.3-20.11 Warning: match nonexhaustive
      0 => ...
      2 => ...

val casedemo = fn : int -> int
```

Pattern matching and the case expression in ML were discussed in Chapter 3.

9.3 Loops and Variations on WHILE

Loops and their use to perform repetitive operations, especially using arrays, have been one of the major features of computer programming since the beginning—computers were in a sense invented to make the task of performing repetitive operations easier and faster. A general form for a loop construct is given by the structure corresponding to Dijkstra’s guarded if, namely, the **guarded do**:

```
do B1 - > S1
  | B2 - > S2
  | B3 - > S3
  . . .
  | Bn -> Sn
od
```

This statement is repeated until all the B_i ’s are false. At each step, one B_i ’s is selected nondeterministically, and the corresponding S_i is executed.

The most basic form of loop construct, which is essentially a guarded do with only one guard (thus eliminating the nondeterminism), is the while-loop of C/C++/Java,

```
while (e) S
```

or the while-loop of Ada,

```
while e loop S1 ... Sn end loop;
```

In these statements, e (the test expression, which must be Boolean in Ada and Java, but not C or C++) is evaluated first. If it is true (or nonzero), then the statement S (or statement sequence $S1 \dots Sn$) is executed, then e is evaluated again, and so on. Note that if e is false (or zero) to begin with, then the code inside is never executed. Most languages have an alternative statement that ensures that the code of the loop is executed at least once. In C and Java, this is the do (or do-while statement):

```
do S while (e);
```

Of course, this is exactly equivalent to the following code,

```
S;
while (e) S
```

so the do-statement is “**syntactic sugar**” (a language construct that is completely expressible in terms of other constructs), but it does express a common situation.

The **while** and **do** constructs have the property that termination of the loop is explicitly specified only at the beginning (while-statement) or end (do-statement) of the loop. Often it is also convenient to exit from inside a loop at one or more intermediate points. For this reason, C (and Java) provide two options. The first option is a **break** statement, which can be used inside a loop (in addition to its use inside a **switch** statement) to exit the loop completely (and continue execution with the statement immediately following the loop). The second option is a **continue** statement that skips the remainder of the body of the loop, resuming execution with the next evaluation of the control expression. The **break** statement is called an **exit** statement in Ada. Ada has no **continue** statement.

Multiple termination points complicate the semantics of loops, so some languages (like Pascal) prohibit them. A typical idiom in C is to indicate the presence of internal termination points (which might otherwise be overlooked) by writing 1 (or `true` in C++ or Java) for the test in a while-loop:

```
while (1) /* always succeeds */
{ ...
  if ( ... ) break;
}
```

This same effect is achieved in Ada by simply omitting the `while` at the beginning of a loop:

```
loop S1... exit when e; ... Sn end loop;
```

A common special case of a looping construct is the **for-loop** of C/C++/Java:

```
for ( e1; e2; e3 ) S;
```

This is completely equivalent in C to:

```
e1;
while (e2)
{ S;
  e3;
}
```

The expression `e1` is the **initializer** of the for-loop, `e2` is the **test**, and `e3` is the **update**. Typically, a for-loop is used in situations where we want an index to run through a set of values from first to last, as in processing the elements of an array:

```
for (i = 0; i < size; i++)
    sum += a[i];
```

Note that the **index** variable `i` is declared outside of the for-loop. C++, Java, and the 1999 ISO C Standard offer the additional feature that a for-loop initializer may contain declarations, so that a loop index can be defined right in the loop:

```
for ( int i = 0; i < size; i++)
    sum += a[i];
```

The scope of such declarations is limited to the rest of the loop; it is as if the for-loop code were inside its own block:⁶

⁶Many C++ compilers as of this writing do not implement this correctly.

```
{ // for loop in C++, Java, and 1999 ISO C
  e1; // may contain local declarations
  while (e2)
  { S;
    e3;
  }
}
```

Many languages severely restrict the format of the for-loop (which, as noted, is just syntactic sugar in C), so that it can *only* be used for indexing situations. For example, in Ada the for-loop can *only* be written as a process controlled by a single local index variable, which is implicitly defined by the loop. Thus, the previous C for-loop would be written in Ada as:

```
for i in 0 .. size - 1 loop
  sum := sum + a(i);
end loop;
```

and *i* is implicitly defined and is local to the code of the loop. Even more, *i* is viewed as a *constant* within the body of the loop, so assignments to *i* are illegal (because this would disrupt the normal loop progression):

```
for i in 0 .. size - 1 loop
  sum := sum + a(i);
  if sum > 42 then i := size; -- illegal in Ada
  end if;
end loop;
```

This form of loop is often included in languages because it can be more effectively optimized than other loop constructs. For example, the control variable (and perhaps the final bound) can be put into registers, allowing extremely fast operations. Also, many processors have a single instruction that can increment a register, test it, and branch, so that the loop control and increment can occur in a single machine instruction. To gain this efficiency, however, many restrictions must be enforced, such as those for Ada mentioned above. Most of the restrictions involve the control variable *i*. Typical restrictions are the following:

- The value of *i* cannot be changed within the body of the loop.
- The value of *i* is undefined after the loop terminates.
- *i* must be of restricted type and may not be declared in certain ways, for example, as a parameter to a procedure, or as a record field, or perhaps it must even be a local variable.

Further questions about the behavior of loops include:

- Are the bounds evaluated only once? If so, then the bounds may not change after execution begins.

- If the lower bound is greater than the upper bound, is the loop executed at all? Most modern languages perform a bound test at the beginning rather than the end of the loop, so the loop behaves like a while-loop. Some older FORTRAN implementations, however, have loops that always execute at least once.
- Is the control variable value undefined even if an `exit` or `break` statement is used to leave the loop before termination? Some languages permit the value to be available on “abnormal termination,” but not otherwise.
- What translator checks are performed on loop structures? Some language definitions do not insist that a translator catch assignments to the control variable, which can cause unpredictable results.

As you saw in Chapter 5, object-oriented languages provide a special type of object called an iterator for looping over the elements of a collection. At a minimum, an iterator includes methods for testing for the presence of an element in the sequence, moving to the next element, and accessing the element just visited. Such languages often use an iterator as the underlying mechanism of a `for - each` loop, which allows the programmer to access all of the elements in a collection with very little syntactic overhead. Figure 9.8 shows two functionally equivalent Java code segments that use an iterator to visit and print all the elements in a list of strings. The first one uses an iterator and its methods explicitly, whereas the second one uses the simpler `for` loop syntax.

```
Iterator iter<String> = list.iterator();
while (iter.hasNext())
    System.out.println(iter.next());

for (String s : list)
    System.out.println(s);
```

Figure 9.8 Two ways to use a Java iterator to process a list

9.4 The GOTO Controversy and Loop Exits

Gotos used to be the mainstay of a number of programming languages, such as Fortran77 and BASIC. For example, the Fortran77 code:

```
10 IF (A(I).EQ.0) GOTO 20
...
I = I + 1
GOTO 10
20 CONTINUE
```

is the Fortran77 equivalent of the C:

```
while (a[i] != 0) i++;
```

With the increasing use of structured control in the 1960s, as pioneered by Algol60, computer scientists began a debate about the usefulness of `gotos`, since they can so easily lead to unreadable **spaghetti code**, as in Figure 9.9.

```

      IF (X.GT.0) GOTO 10
      IF (X.LT.0) GOTO 20
      X = 1
      GOTO 30
10    X = X + 1
      GOTO 30
20    X = -X
      GOTO 10
30    CONTINUE

```

Figure 9.9 An example of spaghetti code in Fortran77

Then, in 1966, Böhm and Jacopini produced the theoretical result that `gotos` were in fact *completely unnecessary*, since *every* possible control structure could be expressed using structured control constructs (although of course with the possible addition of a significant number of extra tests). Finally, in 1968, the computer scientist E. W. Dijkstra published a famous note (“GOTO Statement Considered Harmful,” Dijkstra [1968]) in which he forcefully argued that not only were `goto` statements unnecessary but that their use was damaging in many ways.

The `goto` statement is very close to actual machine code. As Dijkstra pointed out, its “unbridled” use can compromise even the most careful language design and lead to undecipherable programs. Dijkstra proposed that its use be severely controlled or even abolished. This unleashed one of the most persistent controversies in programming, which raged up until the late 1980s (and even occasionally causes outbursts to this day). At first, a large number of programmers simply refused to believe the theoretical result and considered programming without `gotos` to be impossible. However, the academic community began campaigning vigorously for Dijkstra’s position, teaching `goto`-less programming whenever possible. Thanks to these efforts, along with the increasing influence of Pascal, which applies strong limits on the use of `gotos`, programmers gradually became used to the idea of only using `gotos` in rare situations in which the flow of control seems difficult to write in a natural way using standard loops and tests.

However, programmers continued to cling to the idea that the use of `gotos` *was* justified in certain cases, and that academicians were doing a disservice to the programming community by not teaching the “proper” use of `gotos`. This view was succinctly expressed in a letter in 1987 (“‘Goto considered harmful’ considered harmful,” Rubin [1987]) which unleashed a new flurry of controversy over whether there was sufficient justification for the complete abolition of `goto` statements in high-level languages.

Nowadays, depending on the audience and point of view, this controversy may seem quaint. However, echoes of the `goto` controversy continue to be heard. For example, there is still some debate on the propriety of unstructured exits from loops. Proponents of structured exits argue that a loop should always have a single exit point, which is either at the beginning point of the loop (in the header of a `while` loop or a `for` loop) or at the end of the loop (in the footer of a `do-while` or `repeat-until` loop). In a language without a `goto` or similar construct, the syntax of high-level loop structures both supports and enforces this restriction. The opponents of this orthodoxy, typically programmers who use

C, C++, Java, or Python, argue that this restriction forces the programmer to code solutions to certain problems that are more complicated than they would be if unstructured exits were allowed. Two of these problems are the search of an array for a given element and the use of a sentinel-based loop to input and process data. To see the issues involved, we present Java code examples that solve each problem using the two competing approaches.

A typical search method expects an array of integers and a target integer as parameters. The method returns the index of the target integer if it's in the array, or -1 otherwise. Table 9.1 shows two versions of this method, using structured and unstructured exits from the loop.

Table 9.1 Search methods using structured and unstructured loop exits	
Search Using Structured Loop Exit	Search Using Unstructured Loop Exit
<pre>int search(int array[], int target){ boolean found = false; int index = 0; while (index < array.length && ! found) if (array[index] == target) found = true; else index++; if (found) return index; else return -1; }</pre>	<pre>int search(int array[], int target){ for (int index = 0; index < array.length; index++) if (array[index] == target) return index; return -1; }</pre>

As you can see, the first method in Table 9.1 uses a `while` loop with two conditions, one of which tests for the end of the array. The other tests for the detection of the target item. This version of the method requires a separate Boolean flag, as well as a trailing `if-else` statement to return the appropriate value. The method adheres to the structured loop exit principle, but at the cost of some complexity in the code. The second version of the method, by contrast, uses an embedded `return` statement to exit a `for` loop immediately upon finding the target value. If the target is not found, the loop runs to its structured termination, and a default of -1 is returned below the loop. The unstructured loop exit, thus, eliminates the need for a Boolean flag and a trailing `if-else` statement.

A sentinel-based loop is often used in situations where a series of input values must be processed. Table 9.2 shows two versions of a Java method that accepts integers from a scanner object and processes them. The sentinel value to terminate the process is -1.

Table 9.2 Methods using structured and unstructured sentinel-based loop exits	
Structured Loop Exit	Unstructured Loop Exit
<pre>void processInputs(Scanner s){ int datum = s.nextInt(); while (datum != -1){ process(datum); datum = s.nextInt(); } }</pre>	<pre>void processInputs(Scanner s){ while (true){ int datum = s.nextInt(); if (datum == -1) break; process(datum); } }</pre>

The first method in Table 9.2 shows why this problem is often called the **loop and a half problem**. To maintain a structured loop exit, a separate priming input statement must read the first datum above the loop. Another input statement is run at the bottom of the loop before the datum is tested for the sentinel value once more. The second version of the method eliminates the redundant input statement by placing the single input statement within a `while (true)` loop. The input statement is immediately followed by an `if` statement that tests for the sentinel. A nested `break` statement exits the loop if the sentinel is reached. Although this code includes an extra `if` statement, there is now just a single input statement, and the loop control condition is as simple as it could be.

These examples will likely not end the GOTO debate; for an extended discussion, see Roberts [1995].

9.5 Exception Handling

So far, all the control mechanisms you have studied have been **explicit**. At the point where a transfer of control takes place, there is a syntactic indication of the transfer. For example, in a **while**-loop, the loop begins with the keyword `while`. In a procedure call, the called procedure with its arguments is named at the point of call. There are situations, however, where transfer of control is **implicit**, which means the transfer is set up at a different point in the program than that where the actual transfer takes place. At the point where the transfer actually occurs, there may be no syntactic indication that control will in fact be transferred.

Such a situation is **exception handling**, which is the control of error conditions or other unusual events during the execution of a program. Exception handling involves the declaration of both exceptions and exception handlers. An exception is any unexpected or infrequent event. When an exception occurs, it is said to be **raised** or **thrown**. Typical examples of exceptions include runtime errors, such as out-of-range array subscripts or division by zero. In interpreted languages, exceptions can also include static errors, such as syntax and type errors. (These errors are not exceptions for compiled languages, since a program containing them cannot be executed.) Exceptions need not be restricted to errors, however. An exception can be any unusual event, such as input failure or a timeout. An **exception handler** is a procedure or code sequence that is designed to be executed when a particular exception is raised and that is supposed to make it possible for normal execution to continue in some way. An exception handler is said to **handle** or **catch** an exception.

Exception handling was pioneered by the language PL/I in the 1960s and significantly advanced by CLU in the 1970s, with the major design questions eventually resolved in the 1980s and early 1990s. Today, virtually all major current languages, including C++, Java, Ada, Python, ML, and Common Lisp (but not C, Scheme, or Smalltalk) have built-in exception-handling mechanisms. Exception handling has, in particular, been integrated very well into object-oriented mechanisms in Python, Java, and C++, and into functional mechanisms in ML and Common Lisp. Also, languages that do not have built-in mechanisms sometimes have libraries available that provide them, or have other built-in ways of simulating them. Still, exception-handling techniques are often ignored when programming is taught and deserve more widespread use by working programmers.

In this chapter, we will use C++ as our main example for exception handling, with additional overviews of Ada and ML exceptions.

Exception handling is an attempt to imitate in a programming language the features of a hardware interrupt or error trap, in which the processor transfers control automatically to a location that is specified

in advance according to the kind of error or interrupt. It is reasonable to try to build such a feature into a language, since it is often unacceptable for a program to allow the underlying machine or operating system to take control. This usually means that the program is aborted, or “crashes.” Programs that exhibit this behavior fail the test of **robustness**, which is part of the design criteria of security and reliability. A program must be able to recover from errors and continue execution.

Even in a language with a good exception mechanism, however, it is unreasonable to expect a program to be able to catch and handle every possible error that can occur. The reason is that too many possible failures can occur at too low a level—the failure of a hardware device, memory allocation problems, communication problems—all can lead to situations in which the underlying operating system may need to take drastic action to terminate a program, without the program being able to do anything about it. Such errors are sometimes referred to as **asynchronous** exceptions, since they can occur at any moment, not just in response to the execution of program code. By contrast, errors that a program can definitely catch are called **synchronous** exceptions and are exceptions that occur in direct response to actions by the program (such as trying to open a file, perform a particular calculation, or the like). User-defined exceptions can only be synchronous (since they must be raised directly in program code), but predefined or library exceptions may include a number of asynchronous exceptions, since the runtime environment for the language can cooperate with the operating system to allow a program to trap some asynchronous errors.

It is helpful in studying exception-handling mechanisms to recall how exceptions can be dealt with in a language without such facilities. Exception conditions in such languages have to be found before an error occurs, and this assumes it is possible to test for them in the language. One can then attempt to handle the error at the location where it occurs, as in the following C code:

```
if (y == 0)
    handleError("denominator in ratio is zero");
else
    ratio = x / y;
```

Or, if the error occurs in a procedure, one can either pass an error condition back to the caller, as in the C code

```
enum ErrorKind {OutOfInput, BadChar, Normal};
...
ErrorKind getNumber ( unsigned* result)
{ int ch = fgetc(input);
  if (ch == EOF) return OutOfInput;
  else if (! isdigit(ch)) return BadChar;
  /* continue to compute */
  ...
  *result = ... ;
  return Normal;
}
```

or, alternatively, one could pass an exception-handling procedure into the procedure as a parameter:

```
enum ErrorKind {OutOfInput, BadChar, Normal};
typedef void (*ErrorProc)(ErrorKind);
...
unsigned value;
...
void handler (ErrorKind error)
{
    ...
}
...
unsigned getNumber (ErrorProc handle)
{ unsigned result;
  int ch = fgetc(input);
  if (ch == EOF) handle(OutOfInput);
  else if (! isdigit(ch)) handle(BadChar);
  /* continue to compute */
  ...
  return result;
}
...
...
value = getNumber (handler);
```

Explicit exception testing makes a program more difficult to write, since the programmer must test in advance for all possible exceptional conditions. We would like to make this task easier by declaring exceptions in advance of their occurrence and specifying what a program is to do if an exception occurs. In designing such program behavior we must consider the following questions:

Exceptions. What exceptions are predefined in the language? Can they be disabled? Can user-defined exceptions be created? What is their scope?

Exception handlers. How are they defined? What is their scope? What default handlers are provided for predefined exceptions? Can they be replaced?

Control. How is control passed to a handler? Where does control pass after a handler is executed? What runtime environment remains after an error handler executes?

We will discuss each of these topics in turn with examples from C++, Ada, and ML.

9.5.1 Exceptions

Typically an exception occurrence in a program is represented by a data object that may be either predefined or user-defined. Not surprisingly, in a functional language this data object is a *value* of some type, while in a structured or object-oriented language it is a variable or object of some structured type. Typically, the *type* of an exception value or object is predefined, with exceptions themselves given in

a special declaration that creates a value or variable of the appropriate type. For example, in ML or Ada an exception is declared using the reserved word `exception`:

```
exception Trouble; (* a user-defined exception *)
exception Big_Trouble; (* another user-defined exception *)
```

Unusually, in C++ there is no special exception type, and thus no reserved word to declare them. Instead, any structured type (`struct` or `class`) may be used to represent an exception:⁷

```
struct Trouble {} trouble;
struct Big_Trouble {} big_trouble;
```

Now, these declarations as shown are minimal, since they only provide simple values or objects that represent the occurrence of an exception. Typically, we would want also some additional information to be attached to these exceptions, such as an error message (a string) and also perhaps a summary of the data or program state that led to the exception (such as a numeric value that led to an invalid computation, or an input character that was unexpected). It is easy to see how to do this in C++. We simply add data members to the defined data structure:

```
struct Trouble{
    string error_message;
    int wrong_value;
} trouble;
```

In ML this can also be done:

```
exception Trouble of string * int;
```

Unusually, in Ada this cannot be done directly. Exception objects are constants that contain no data.⁸

Exception declarations such as the above typically observe the same scope rules as other declarations in the language. Thus, in Ada, ML, and C++, lexical, or static, scope rules are observed for exception declarations. Since exceptions occur during runtime and, as you will see below, can cause execution to exit the scope of a particular exception declaration, it can happen that an exception cannot be referred to by name when it is handled. The handler's design must take this into account. Typically, one wishes to minimize the trouble that this causes by declaring user-defined exceptions globally in a program. Local exceptions may under certain circumstances make sense, however, to avoid creating a large number of superfluous global exceptions. Note also that, when data are included in an exception, we want to separate the exception *type* declaration from the exception *object/value* declaration, since in general different instances of the same exception type may exist simultaneously with different data, so the objects/values must be created as the exceptions occur. As long as the exception types themselves are global, no scope issues result, as you will see below. Thus, the C++ declaration above (which defines a variable as well as a type) should be amended to only declare a type:

⁷Since we do not discuss the object-oriented features of C++ until Chapter 11, we write C++ code that is as close to C as possible in this section, even though somewhat unidiomatic.

⁸Ada95 does add a library `Ada.Exceptions` that has mechanisms for passing information along with exceptions, but it is not as clean as adding data to the exception objects themselves.

```
struct Trouble{
    string error_message;
    int wrong_value;
} ; // declare exception object later
```

Most languages also provide some predefined exception values or types. For instance, in Ada there are the following predefined exceptions:

Constraint_Error: Caused by exceeding the bounds of a subrange or array; also caused by arithmetic overflow and division by zero.

Program_Error: This includes errors that occur during the dynamic processing of a declaration.

Storage_Error: This is caused by the failure of dynamic memory allocation.

Tasking_Error: This occurs during concurrency control.

In ML there are a number of similar predefined exceptions, such as `Div` (division by 0), `Overflow` (arithmetic), `Size` (memory allocation), and `Subscript` (array subscript out of bounds); additional predefined exceptions exist that have to do with specific program constructs in ML and that correspond roughly to Ada's `Program_Error`. There also is a predefined value constructor `Fail of string`, which allows a program to construct exception values containing an error message, without having to define a new exception type.

In C++, in keeping with usual practice, there are no predefined exception types or exceptions in the language proper. However, many standard library modules provide exceptions and exception mechanisms. Some of these standard exceptions are:

bad_alloc: Failure of call to `new`.

bad_cast: Failure of a `dynamic_cast` (see Chapter 5).

out_of_range: Caused by a checked subscript operation on library containers (the usual subscript operation is not checked).

overflow_error, underflow_error, range_error: Caused by math library functions and a few others.

We also mention below another standard exception, `bad_exception`, which can be useful in handling exceptions in C++.

9.5.2 Exception Handlers

In C++ exception handlers are associated with **try-catch** blocks, which can appear anywhere a statement can appear. A sketch of a C++ try-catch block appears in Figure 9.10.

```
(1) try
(2) { // to perform some processing
(3)   ...
(4) }
(5) catch (Trouble t)
(6) { // handle the trouble, if possible
```

Figure 9.10 A C++ try-catch block (*continues*)

(continued)

```

(7)    displayMessage(t.error_message);
(8)    ...
(9)   }
(10)  catch (Big_Trouble b)
(11)  { // handle big trouble, if possible
(12)    ...
(13)  }
(14)  catch (...) // actually three dots here, not an ellipsis!
(15)  { // handle any remaining uncaught exceptions
(16)  }

```

Figure 9.10 A C++ try-catch block

Here the compound statement after the reserved word `try` is required (the curly brackets on lines 2 and 4 of Figure 9.10), and any number of `catch` blocks can follow the initial `try` block. Each `catch` block also requires a compound statement and is written as a function of a single parameter whose type is an exception type (which can be any class or structure type). The exception parameter may be consulted to retrieve appropriate exception information, as in line 7 of Figure 9.10. Note also in Figure 9.10 the catch-all `catch` block at the end that catches any remaining exceptions, as indicated by the three dots inside the parentheses on line 14.

The syntax of try-catch blocks in C++ is really overkill, since `catch` blocks could have been associated with any statement or block. Indeed, in Ada and ML that is exactly the case. For example, in Ada the reserved word `exception` is reused to introduce handlers. Handles can appear at the end of any block, as shown in Figure 9.11, which contains Ada code corresponding to the C++ code of Figure 9.10. Note also the similarity of the syntax of the exception list (lines 5–13 of Figure 9.11) to that of the Ada case statement.

```

(1)  begin
(2)    -- try to perform some processing
(3)    ...
(4)  exception
(5)    when Trouble =>
(6)      --handle trouble, if possible
(7)      displayMessage("trouble here!");
(8)      ...
(9)    when Big_Trouble =>
(10)     -- handle big trouble, if possible
(11)     ...
(12)    when others =>
(13)     -- handle any remaining uncaught exceptions
(14)  end;

```

Figure 9.11 An Ada try-catch block corresponding to Figure 9.9

In ML, the reserved word `handle` can come at the end of any expression, and introduces the handlers for exceptions generated by that expression, as shown in Figure 9.12.

```
(1) val try_to_stay_out_of_trouble =
(2) (* try to compute some value *)
(3) handle
(4)   Trouble (message,value) =>
(5)     ( displayMessage(message); ... ) |
(6)   Big_Trouble => ... |
(7)   _ =>
(8)     (* handle any remaining uncaught exceptions *)
(9)     ...
(10) ;
```

Figure 9.12 An example of ML exception handling corresponding to Figures 9.10 and 9.11

Exceptions in ML are distinguished by pattern-matching/unification within the `handle` clause (see Chapters 3 and 8). The syntax is again similar to that of the `case` expression in ML (Section 9.2, Figure 9.7). In particular, the underscore character on line 7 of Figure 9.12 is again a wildcard pattern as in Figure 9.7 and has the same effect as the Ada `others` clause or the C++ `catch( ... )`. It will match any exception not previously handled.

The scope of handlers defined as above extends, of course, only to the statements/expression to which they are attached. If an exception reaches such a handler, it replaces whatever handlers may exist elsewhere, including predefined handlers.

Predefined handlers typically simply print a minimal error message, indicating the kind of exception, possibly along with some information on the program location where it occurred, and terminate the program. None of the three languages we are considering here require any additional behavior. Also, in Ada and ML there is no way to change the behavior of default handlers, except to write a handler as we have just indicated.

C++, on the other hand, offers a way to replace the default handler with a user-defined handler, using the `<exceptions>` standard library module. One does this by calling the `set_terminate` function, passing it an argument consisting of a `void` function taking no parameters. This function will then be called as the default handler for all exceptions not explicitly caught in the program code.

In Ada it is not possible to replace the default handling action with a different one, but it is possible to *disable* the default handler by telling the compiler not to include code to perform certain checks, such as bounds or range checking. This is done using the `suppress pragma` (compiler directive), as in:

```
pragma suppress(Overflow_Check, On => Float);
```

which turns off all overflow checking for floating-point numbers throughout this pragma's scope (such a pragma is viewed as a declaration with standard scope rules). This can mean that such an Ada program runs to completion and produces incorrect results without any error message being generated. Normally, this is precisely the kind of program behavior that Ada was designed to prevent. However, in those cases where maximum efficiency is needed and the programmers are certain of the impossibility of an exception occurring, this feature can be helpful.

9.5.3 Control

Finally, in this subsection we discuss how exceptions are reported and how control may be passed to a handler. Typically, a predefined or built-in exception is either automatically raised by the runtime system, or it can be manually raised by the program. User-defined exceptions can, of course, only be raised manually by the program.

C++ uses the reserved word `throw` and an exception object to raise an exception:

```
if (/* something goes wrong */)
{ Trouble t; // create a new Trouble var to hold info
  t.error_message = "Bad news!";
  t.wrong_value = current_item;
  throw t;
}
else if (/* something even worse happens */)
  throw big_trouble; // can use global var, since no info
```

Ada and ML both use the reserved word `raise`:

```
-- Ada code:
if -- something goes wrong
then
  raise Trouble; -- use Trouble as a constant
elsif -- something even worse happens
then
  raise Big_Trouble; -- use Big_Trouble as a constant
end if;

(* ML code: *)
if (* something goes wrong *)
then (* construct a Trouble value *)
  raise Trouble("Bad news!",current_item)
else if (* something even worse happens *)
  raise Big_Trouble (* Big_Trouble is a constant *)
else ... ;
```

When an exception is raised, typically the current computation is abandoned, and the runtime system begins a search for a handler. In Ada and C++, this search begins with the handler section of the block in which the exception was raised. If no handler is found, then the handler section of the next enclosing block is consulted, and so on (this process is called **propagating the exception**). If the runtime system reaches the outermost block of a function or procedure without finding a handler, then the call is exited to the caller, and the exception is again raised in the caller as though the call itself had raised the exception. This proceeds until either a handler is found, or the main program is exited, in which case the default handler is called. The process is similar in ML, with expressions and subexpressions replacing blocks.

The process of exiting back through function calls to the caller during the search for a handler is called **call unwinding** or **stack unwinding**, the stack being the call stack whose structure is described in detail in the next chapter.

Once a handler has been found and executed, there remains the question of where to continue execution. One choice is to return to the point at which the exception was first raised and begin execution again with that statement or expression. This is called the **resumption model** of exception handling, and it requires that, during the search for a handler and possible call unwinding, the original environment and call structure must be preserved and reestablished prior to the resumption of execution.

The alternative to the resumption model is to continue execution with the code immediately following the block or expression in which the handler that is executed was found. This is called the **termination model** of exception handling, since it essentially discards all the unwound calls and blocks until a handler is found.

Virtually all modern languages, including C++, ML, and Ada, use the termination model of exception handling. The reasons are not obvious, but experience has shown that termination is easier to implement and fits better into structured programming techniques, while resumption is rarely needed and can be sufficiently simulated by the termination model when it is needed.

Here is an example of how to simulate the resumption model using termination in C++. Suppose that a call to `new` failed because there was not enough free memory left, and we wished to call a garbage collector and then try again. We could write the code as follows:

```
while (true)
    try
    { x = new X; // try to allocate
      break; // if we get here, success!
    }
    catch (bad_alloc)
    { collect_garbage(); // can't exit yet!
      if ( /* still not enough memory */ )
        // must give up to avoid infinite loop
        throw bad_alloc;
    }
}
```

Finally, we mention also that exception handling often carries substantial runtime overhead (see the next chapter for details). For that reason, and also because exceptions represent a not very structured control alternative, we want to avoid overusing exceptions to implement ordinary control situations, where simple tests would do the job. For example, given a binary search tree data structure (containing, e.g., integers), such as

```
struct Tree{
    int data;
    Tree * left;
    Tree * right;
};
```

we could write a search routine as in Figure 9.13.

```
void fnd (Tree* p, int i) // helper procedure
{ if (p != 0)
    if (i == p->data) throw p;
    else if (i < p->data) fnd(p->left,i);
    else fnd(p->right,i);
}

Tree * find (Tree* p, int i)
{ try
    { fnd(p,i);
    }
    catch(Tree* q)
    { return q;
    }
    return 0;
}
```

Figure 9.13 A binary tree find function in C++ using exceptions (adapted from Stroustrup [1997], pp. 374–375)

However, this code is likely to run much more slowly than a more standard search routine that does not use exceptions. Also, more standard code is probably easier to understand.

9.6 Case Study: Computing the Values of Static Expressions in TinyAda

In languages such as Pascal and Ada, a symbolic constant has a value that can be determined at compile time. Pascal requires its symbolic constants to be defined as literals. Ada is more flexible, allowing the value of any expression not including a variable or a function call to serve as the value of a symbolic constant. Such expressions are called **static expressions**, to distinguish them from other types of expressions whose values cannot be known until run time. In this section, we examine how a parser for TinyAda can compute the values of static expressions.

9.6.1 The Syntax and Semantics of Static Expressions

Static expressions can appear in two types of TinyAda declarations. The first, a number declaration, defines a symbolic constant. The second, a range type definition, defines a new subrange type. The next code segment, which defines a type for rectangular matrices whose widths are twice that of their heights, shows these two syntactic forms in action:

```
ROW_MAX : constant := 10;
COLUMN_MAX : constant := ROW_MAX * 2;
type MATRIX_TYPE is range 1..ROW_MAX, range 1..COLUMN_MAX of BOOLEAN;
```

Note how the use of the expression `ROW_MAX * 2` in the definition of `COLUMN_MAX` maintains a 2:1 aspect ratio of the matrix dimensions, allowing the programmer to vary their size by changing just the value of `ROW_MAX`.

The following grammar rules for the two types of declarations show the placements of the static expressions in them:

```
numberDeclaration = identifierList ":" "constant" ":@" <static>expression ";"
```

```
range = "range" <static>simpleExpression ".." <static>simpleExpression
```

Syntactically, static expressions look just like other expressions. Their semantics are also similar, in that the applications of the arithmetic, comparison, and logical operators produce the expected results. However, to ensure that these results can be computed at compile time, static expressions cannot include variable or parameter references.

In TinyAda, the values of static expressions ultimately come from literals in the token stream, or from the values of constant names, including the two built-in symbolic constants, `TRUE` and `FALSE`. The two types of literals include integer and character values. The scanner translates the different source code representations of these to a single internal form—base 10 integers for all of the TinyAda integer representations and ASCII values for the 128 character literals. The internal forms of the two Boolean values are, by convention, the integers 0 for `FALSE` and 1 for `TRUE`. The parser uses these internal representations to evaluate all static expressions, as you will see shortly.

9.6.2 Entering the Values of Symbolic Constants

Each symbolic constant now has a value attribute in its symbol entry record. The method `setValue` installs this value, which must be an integer, for a list of constant identifiers. For example, when symbol entries for the built-in names `TRUE` and `FALSE` are initialized, their values must be entered as well:

```
private void initTable(){
    ...
    entry = table.enterSymbol("TRUE");
    entry.setRole(SymbolEntry.CONST);
    entry.setType(BOOL_TYPE);
    entry.setValue(1);           // TRUE = 1, internally
    entry = table.enterSymbol("FALSE");
    entry.setRole(SymbolEntry.CONST);
    entry.setType(BOOL_TYPE);
    entry.setValue(0);           // FALSE = 0, internally
}
```

The method `numberOrObjectDeclaration` must also install a value for the list of constant identifiers just processed. Working top-down, we assume that the method that parses the TinyAda expression

on the right side of the `:=` symbol also computes this value and returns it. However, there are three problems with simply reusing the current parsing method expression:

1. All expression methods already return a type descriptor, which is still needed for type checking and to set the type attributes of the constant identifiers and the subrange types.
2. These methods permit, through the current chain of calls to lower-level methods, the presence of variable and parameter names, as well as array indexed components.
3. Not all expressions are static, so some control mechanism must be provided to activate the computation of values when needed.

The first problem is easily solved by having each expression method return a symbol entry instead of a type descriptor. When the expression is static, the entry object can contain both the value and the type of the current expression. Each of these attributes is then available for use in further semantic processing.

The other two problems could be solved by passing a Boolean flag as a parameter to each expression method, to control the decisions about whether to compute a value and whether to accept only constant names. However, the additional logic and interface would complicate the implementation. Therefore, we adopt, at the cost of some redundant code, a strategy of defining a second set of methods that parse only static expressions.

Here is the code for `numberOrObjectDeclaration`, which shows a call of the new top-level method `staticExpression`:

```
/*
objectDeclaration = identifierList ":" typeDefinition ";"

numberDeclaration = identifierList ":" "constant" ":@" <static>expression ";"
*/
private void numberOrObjectDeclaration(){
    SymbolEntry list = identifierList();
    accept(Token.COLON, "':' expected");
    if (token.code == Token.CONST){
        list.setRole(SymbolEntry.CONST);
        token = scanner.nextToken();
        accept(Token.GETS, "':'=" expected");
        SymbolEntry entry = staticExpression(); // Get the entry here and
        list.setType(entry.type);              // then extract the type
        list.setValue(entry.value);            // and the value.
    }
    else{
        list.setRole(SymbolEntry.VAR);
        list.setType(typeDefinition());
    }
    accept(Token.SEMI, "semicolon expected");
}
```

The method `range` receives the entries returned by the processing of two static simple expressions. The values in these entries must be installed as the lower and upper bounds of a new subrange type descriptor. The range method performs type checking on the simple expressions, as before, and now can check to see that the lower bound is less than or equal to the upper bound. The changes to the code for the method `range` are left as an exercise for the reader.

9.6.3 Looking Up the Values of Static Expressions

The value of the simplest form of a static expression is encountered in the new method `staticPrimary`. This value will be an integer or character literal in the token stream or the value of a constant identifier. The structure of this method is similar to that of its nonstatic counterpart, with two exceptions. First, the new method returns a symbol entry. Second, the new method simply looks up an identifier rather than calling the method name. Here is the code for the method `staticPrimary`:

```
/*
primary = numericLiteral | name | "(" expression ")"
*/
SymbolEntry staticPrimary(){
    SymbolEntry entry = new SymbolEntry();
    switch (token.code){
        case Token.INT:
            entry.type = INT_TYPE;
            entry.value = token.integer;
            token = scanner.nextToken();
            break;
        case Token.CHAR:
            entry.type = CHAR_TYPE;
            entry.value = token.integer;
            token = scanner.nextToken();
            break;
        case Token.ID:
            entry = findId();
            acceptRole(entry, SymbolEntry.CONST, "constant name expected");
            break;
        case Token.L_PAR:
            token = scanner.nextToken();
            entry = staticExpression();
            accept(Token.R_PAR, "' ' expected");
            break;
        default: fatalError("error in primary");
    }
    return entry;
}
```

Note that the method must create a new symbol entry object to contain the value and type information of a character literal or an integer literal. Otherwise, the entry is either that of the identifier or the one returned by the nested call to `staticExpression`.

9.6.4 Computing the Values of Static Expressions

The other static expression methods also pass a symbol entry object back to their callers. If no operators are encountered, this entry comes from the processing of a single subexpression. Otherwise, we must deal with two or more symbol entries, each of which is the result of parsing an operand expression. The types of these entries are checked, as before. The current operator is then applied to the values of each pair of entries to compute a new value. This value, suitably wrapped in a new symbol entry object, is then either returned or used in further computations.

As an example of this processing, we show the code for the method `staticTerm`, which computes and returns the value of one or more factors separated by multiplying operators. Here is the code, followed by an explanation:

```
/*
term = factor { multiplyingOperator factor }
*/
private SymbolEntry staticTerm(){
    SymbolEntry s1 = staticFactor();
    if (multiplyingOperator.contains(token.code))
        matchTypes(s1.type, INT_TYPE, "integer expression expected");
    while (multiplyingOperator.contains(token.code)){
        Token op = token;
        token = scanner.nextToken();
        SymbolEntry s2 = staticFactor();
        matchTypes(s2.type, INT_TYPE, "integer expression expected");
        s1 = computeValue(s1.value, s2.value, op);
    }
    return s1;
}
```

As before, this method checks the types of the operands only if it sees an operator. Now, however, those types are buried in symbol entries. Moreover, the method must remember the particular operator token just seen, using the temporary variable `op`, so that it can be used to compute the term's value. Finally, the helper method `computeValue` is passed the values of the two operands and the operator as parameters. This method computes the new value, installs it in a new symbol entry object, and returns this object to the `term` method for further processing. The definitions of the `computeValue` method and the remaining methods for parsing static expressions are left as exercises for the reader.

Exercises

- 9.1** Rewrite the following infix expression in prefix and postfix form and draw the syntax tree:

$$(3 - 4) / 5 + 6 * 7$$

- 9.2** Write a BNF description of **(a)** postfix arithmetic expressions and **(b)** prefix arithmetic expressions.
- 9.3** Modify the recursive descent calculator of Figure 6.12 to translate infix expressions to postfix expressions.

- 9.4** In LISP the following unparenthesized prefix expression is ambiguous:

$$+ 3 * 4 5 6$$

Why? Give two possible parenthesized interpretations.

- 9.5** Examples were given in the text that show the usefulness of a short-circuit and operation. Give an example to show the usefulness of a short-circuit or operation.
- 9.6** Write a program to prove that short-circuit evaluation is used for the logical operators of C/C++.
- 9.7** Write a program to determine the order of evaluation of function arguments for your C/C++/Ada compiler.
- 9.8** Java specifies that all expressions, including arguments to calls, are evaluated from left to right. Why does Java do this, while C/C++ does not?
- 9.9** We noted in Chapter 5 that the $+$ operator is left associative, so that in the expression $a + b + c$, the expression $a + b$ is computed and then added to c . Yet we stated in Section 9.1 that an expression $a + b$ may be computed by computing b before a . Is there a contradiction here? Does your answer also apply to the subtraction operator? Explain.
- 9.10** Suppose that we were to try to write a short-circuit version of `and` in C as the following function:

```
int and (int a, int b){
    return a ? b : 0
}
```

- (a)** Why doesn't this work?
- (b)** Would it work if normal order evaluation were used? Why?
- 9.11** Describe one benefit of normal order evaluation. Describe one drawback.
- 9.12** Consider the expressions

$$(x \neq 0) \text{ and } (y \% x == 0)$$

and

$$(y \% x == 0) \text{ and } (x \neq 0)$$

In theory, both these expressions could have the value `false` if $x == 0$.

- (a)** Which of these expressions has a value in C?
- (b)** Which of these expressions has a value in Ada?
- (c)** Would it be possible for a programming language to require that both have values? Explain.

- 9.13** Describe the difference between the C/C++ operators `&&` and `||` and the operators `&` and `|`. Are these latter operators short-circuit? Why or why not?

- 9.14** Given the two functions (in C syntax):

```
int cube(int x){ return x*x*x; }
int sum(int x, int y, int z){ return x + y + z; }
```

describe the process of normal order evaluation of the expression

`sum(cube(2),cube(3),cube(4))`, and compare it to applicative order evaluation. In particular, how many additions and multiplications are performed using each method?

- 9.15** A problem exists in normal order evaluation of recursive functions. Describe the problem using as an example the recursive factorial function

```
int factorial(int n)
{ if (n == 0) return 1; else return n * factorial ( n - 1 ); }
```

Propose a possible solution to the problem, and illustrate it with the above function, using the call `factorial(3)`.

- 9.16** Describe the process of evaluating the expression `factorial(5)` using normal order. When are the subtractions (such as `5 - 1`) performed? When are the multiplications (such as `5 * 24`) performed?

- 9.17** C insists that a sequence of statements be surrounded by braces `{ ... }` in structured statements such as while-statements:

```
while-stmt → while ( expression ) statement
statement → compound-statement | ...
compound-statement → { [ declaration-list ] [ statement-list ] }
statement-list → statement-list statement | statement
```

Suppose that we eliminated the braces in compound statements and wrote the grammar as follows:

```
while-stmt → while ( expression ) statement
statement → compound-statement | ...
compound-statement → [ declaration-list ] [ statement-list ]
statement-list → statement-list statement | statement
```

Show that this grammar is ambiguous. What can be done to correct it without going back to the C convention?

- 9.18** The default case in a switch statement in C/C++ corresponds in a way to the else-part of an if-statement. Is there a corresponding “dangling-default” ambiguity similar to the dangling-else ambiguity in C/C++? Explain.
- 9.19** Since the default case in a switch statement in C/C++ is similar to the else-part of an if-statement, why not use the reserved word `else` instead of the new keyword `default`? What design principles apply here?
- 9.20** Show how to imitate a while-statement in C/C++ with a do-statement.
- 9.21** Show how to imitate a for-statement in C/C++ with a do-statement.
- 9.22** Show how to imitate a while-statement in C/C++ with a for-statement.

- 9.23** Show how to imitate a do-statement in C/C++ with a for-statement.
- 9.24** You saw in this chapter (and in Exercise 8.14) that it makes sense to have an if-expression in a language.
- (a) Does it make sense to have a while-expression; that is, can a while-expression return a value, and if so, what value?
 - (b) Does it make sense to have a case-or switch-expression; that is, can a case-expression return a value, and if so, what value?
 - (c) Does it make sense to have a do-or repeat-expression; that is, can a do-expression return a value, and if so, what value?
- 9.25** We noted in the text that, in a language that uses normal order evaluation, an if-expression can be written as an ordinary function. Can a while-expression be written in such a language? Explain.
- 9.26** Give examples of situations in which it would be appropriate to use a break statement and a return statement to exit loops.
- 9.27** An important difference between the semantics of the case-statement in C/C++ and Ada is that an unlisted case causes a runtime error in Ada, while in C/C++, execution “falls through” to the statement following the case-statement. Compare these two conventions with respect to the design principles of Chapter 2. Which principles apply?
- 9.28** (Aho, Sethi, and Ullman [1986]) The problem with the dangling else in C/C++ can be fixed by writing more complicated syntax rules. One attempt to do so might be as follows:
- statement* $\rightarrow$ *if (expression) statement | matched-statement*
matched-statement $\rightarrow$
 if (expression) matched-statement else statement | ...
- (a) Show that this grammar is still ambiguous.
 - (b) How can it be fixed so that the grammar expresses the most closely nested rule unambiguously?
- 9.29** Locate a grammar for the Java language, and describe how it fixes the dangling-else ambiguity. How many extra grammar rules does it use to do this?
- 9.30** Here are three possible switch-statements in C/C++/Java syntax:

```
int x = 2;
switch (x) x++;

int x = 2;
switch (x)
{ x ++; }

int x = 2;
switch (x)
{ case 1: if (x > 2)
  case 2:  x++;
  default: break;
}
```

- (a) Which of these are legal in C/C++? For those that are legal, what is the value of x after each is executed?
- (b) Which of these are legal in Java? For those that are legal, what is the value of x after each is executed?

9.31 Here is a legal switch-statement in C/C++:

```
int n = ...;
int q = (n + 3) / 4;
switch (n % 4)
{ case 0: do { n++;
  case 3:   n++;
  case 2:   n++;
  case 1:   n++;
  } while (--q > 0);
}
```

Suppose that n has the value **(a)** 0; **(b)** 1; **(c)** 5; **(d)** 8. What is the value of n in each case after the execution of the above switch-statement? Can you state a general rule for the value of n after this code executes? If so, state it. If not, explain why not.

- 9.32** Describe the effect of the FORTRAN spaghetti code of Figure 9.9, Section 9.4. Write C/C++ statements without GOTOs that are equivalent to the FORTRAN code.
- 9.33** Clark [1973] humorously describes a “come from” statement as a replacement for the GOTO statement:

```
10 J = 1
11 COME FROM 20
12 PRINT *, J
   STOP
13 COME FROM 10
20 J = J + 2
```

This sample program in FORTRAN syntax prints 3 and stops. Develop a description of the semantics of the COME FROM statement. What problems might a translator have generating code for such a statement?

- 9.34** C++ and Java loops are often written with empty bodies by placing all side effects into the tests, such as in the following two examples:

```
i = 0;
while (a[i++] != 0);

for (i = 0; a[i] != 0; i++);
```

- (a) Are these loops equivalent?
- (b) Are there any advantages or disadvantages to using this style of code? Explain.

- 9.35** Test each of the following C/C++ code fragments with a translator of your choice (or rewrite it into another language and test it) and explain its behavior:

(a)

```
int i, n = 3;
for (i = 1; i <= n; i++)
{ printf ("i = %d\n", i);
  n = 2;
}
```

(b)

```
int i;
for (i = 1; i <= 3; i++)
{ printf ("i = %d\n", i);
  i = 3;
}
```

- 9.36** Give an example in C++ or Ada of an exception that is propagated out of its scope.
- 9.37** In Section 9.5, three alternatives to simulating exceptions were discussed: (1) testing for an error condition before the error occurs; (2) passing a flag (an enumerated type) back as well as a value to indicate success or various failure outcomes; or (3) passing an error-handling function as a parameter. A fourth alternative is a variation on (2), where instead of passing two values back, one of which is a flag, we indicate an error by using an actual value, such as -1 or 999 that is not a legal value for the computation but is still a legal value for the result type. Discuss the advantages and disadvantages of this method compared to the use of an enumerated flag. What design principles from Chapter 2 apply?
- 9.38** Suppose that in a C++ `try` block you had some code that you wanted to execute on exit, regardless of whether an exception occurred or not. Where would you have to place this code? What if the code needed to be executed only if an exception did *not* occur?
- 9.39** (a) Rewrite the binary search tree `find` function of Figure 9.13, Section 9.5.3 to eliminate the use of an exception.
 (b) Compare the running times of your version of `find` in part (a) with the version in the text.
- 9.40** Suppose that we wrote the following `try` block in C++:

```
try
{ // do something
}
catch (...) { cout << "general error!\n"; }
catch (range_error) { cout << "range error!\n"; }
```

What is wrong with this code?

9.41 Install the values for the built-in constants `TRUE` and `FALSE` in the `TinyAda` Parser. You should then see these values displayed in the `TinyAda` program listing when you run the parser.

9.42 Add the helper method `computeValue` to the `TinyAda` parser. This method's signature is

```
SymbolEntry computeValue(int v1, int v2, Token op)
```

The method should apply the operator to the two values to compute a new value, and then create and return a new symbol entry object that contains this value and the role of constant. Remember to provide computations for all of the arithmetic, comparison, and logical operators. In the case of the unary operators, you should assume that the first value is 0 and the second value is the actual operand. Remember also that Boolean operands use the representations 0 for `FALSE` and 1 for `TRUE`.

9.43 Add the definitions of the methods for parsing static expressions to the `TinyAda` parser, then modify the definition of the method `numberOrObjectDeclaration` as shown in Section 9.6. Now you can test the parser with some `TinyAda` programs that use static expressions in number declarations.

9.44 Modify the method `range` in the `TinyAda` parser as described in Section 9.6, and test the parser with some `TinyAda` programs that define new subrange types.

Notes and References

Normal order evaluation and applicative order evaluation are described in Abelson and Sussman [1996] and Paulson [1996]. You will see normal order evaluation again in Chapter 10 in the pass by name parameter passing mechanism of Algol60, and you also saw it in Chapter 3 when discussing delayed evaluation and the lambda calculus.

Dijkstra's guarded `if` and guarded `do` commands are described in Dijkstra [1975]. Hoare describes his design of the case-statement in Hoare [1981]. The unusual use of a C switch-statement in Exercise 9.31 is called Duff's device in Gosling et al. [2000, page 289]. The dangling-else ambiguity is discussed in Loudon [1997], Aho, Sethi, and Ullman [1986], and Gosling et al. [2000]; these references describe a number of different approaches to the problem.

Dijkstra's famous letter on the `GOTO` statement appears in Dijkstra [1968a]. A letter by Rubin [1987] rekindled the argument, which continued in the letters of the *Communications of the ACM* throughout most of 1987. Dijkstra's letter, however, was not the earliest mention of the problems of the `GOTO` statement. Naur [1963b] also comments on their drawbacks. For the disciplined use of `GOTO`s, see Knuth [1974]. For an amusing takeoff on the `GOTO` controversy, see Clark [1973] and Exercise 9.33. Roberts's discussion of nonstructured exits from loops and subroutines is in Roberts [1995].

An early seminal paper on exception handling is Goodenough [1975]. Exception handling in CLU, which also was a major influence on modern-day methods, is described and comparisons with other languages are given in Liskov and Snyder [1979]. See also Liskov et al. [1984]. Exception handling in Ada is discussed in Cohen [1996] and in Luckam and Polak [1980]. Exception handling in C++ is described in Stroustrup [1997]; Stroustrup [1994] gives a detailed view of the choices made in designing the exception handling of C++, including an extensive history of termination versus resumption semantics. Exception handling in ML is described in Ullman [1998], Paulson [1996], Milner and Tofte [1991], and Milner et al. [1997]. A control structure related to exceptions is the **continuation** of the Scheme dialect of LISP. For a description of this mechanism see Springer and Friedman [1989], and Friedman, Haynes, and Kohlbecker [1985].